Supervised Learning & Model Engineering

MSA-DATB09-04

Project Instructions

Ilana Sebag

Overview & objectives

Each group builds and defends a complete supervised-learning proof-of-concept end to end, on a tabular binary classification problem: from raw data to a served prediction endpoint. The project mirrors the arc of the course and is worth 50% of the final grade, alongside the written exam.

Team formation

- **Groups of 2–3 students**, self-formed.
- Every member must be able to explain and defend every part of the project.

Technical requirements

Each requirement ties back to a session, use your notes. The following list is not exhaustive, you are expected to identify and define the intermediate steps required to reach the solution by yourselves.

1. **EDA & problem framing**. Understand the dataset and choose an honest train/validation/test.
2. **Feature engineering**. Encode categoricals, handle any missing values, and engineer at least 2-3 features you can justify.
3. **Class imbalance handling**. Report relevant metrics, justify your chosen remedy and show how it was applied in your code.
4. **Modelling progression**. Train and compare at least 3 chosen models. Report the same metrics for all three and discuss what each step bought you.
5. **Pipeline**. Wrap all preprocessing and the model in a single `Pipeline`.
6. **Hyperparameter tuning**. Use Optuna to tune your models, with proper cross-validation so your reported score isn't optimistic.
7. **Experiment tracking**. Log every run, parameters, metrics, and artifacts, for all three models across your tuning trials.
8. **Explainability**. Produce global feature importance and local explanations with SHAP for a handful of individual predictions. Discuss whether the important features make business sense.
9. **Calibration diagnostics**. For your final model, plot a reliability curve and report the Brier score (and/or ECE). State whether the model is over- or under-confident, and whether you'd trust its probabilities for a downstream decision as-is.
10. **Fairness & bias audit**. Using your dataset's sensitive attribute, compute and report relevant metrics, corresponding fairness definition. Apply one mitigation technique and compare performance/fairness gap before/after on the untouched test set.

Datasets

You are free to choose between these three datasets.

```python
from sklearn.datasets import fetch_openml

# Option 1 -- Adult / Census Income
adult = fetch_openml(name="adult", version=2, as_frame=True)
X, y = adult.data, adult.target
```

```python
from sklearn.datasets import fetch_openml

# Option 2 -- Bank Marketing
bank = fetch_openml(data_id=1461, as_frame=True)
X, y = bank.data, bank.target
```

```python
from sklearn.datasets import fetch_openml

# Option 3 -- Diabetes 130 Hospitals
diabetes = fetch_openml(data_id=4541, as_frame=True)
X, y = diabetes.data, diabetes.target
```

Pick **one** of these three datasets. All real, imbalanced, binary classification, and loaded the same way. All three include a genuine sensitive attribute for the Fairness & bias audit requirement.

Don't forget to state which dataset you used at the top of your report!

Deliverables checklist

Deliverable	Detail
Git repository	Clean code including `README.md` and `requirements.txt`. Real commit history, not only the last version of the code!
Written report	∼10 pages excluding appendix, LaTeX. Submit the PDF.
Oral defense	10 minutes: **7 min presentation** + 3 min Q&A.

Code: must be clean and well-organized. The main submission must use Python scripts (.py), together with a requirements.txt, a README.md. If you find relevant, you can use a YAML configuration file. Jupyter notebooks are not accepted as the main implementation and may only be included as **supplementary** material for demonstrations or visualizations only.

Report: is the main place for all explanations, analyses, plots, results, and interpretations. Your code should focus on the implementation, while the report should clearly document and justify your methodological choices and discuss the results.

Contribution statement (uncounted, MANDATORY): a short closing paragraph or table stating who did what, e.g. by technical requirement or pipeline stage, and each member's role.

Grading breakdown

Component	Weight	What it measures
Git technical deliverable	20%	Code quality, correctness, and genuine Git commit history
Technical report	30%	Clear explanation and justification of methodological choices, rigorous analysis of experiments, appropriate presentation of results and plots, and critical interpretation of findings
Group oral defense	40%	Clarity of the presentation and the group's answers in Q&A
Individual deliberation	10%	Each member's own understanding, checked during the oral defense

All group members should expect to speak during the presentation; both the presentation and questions at the end are where personal understanding is checked directly.

Submission instructions & policies

Submission. One group member shares the PDF report that must include the GitHub repository link.

Commit history requirement. The repository must show genuine incremental work: commits spread across the project timeline, meaningful messages, and visible progress through the steps. A single final commit, or commits all dated the day before the deadline, will lose points.

Academic integrity. Work is per-group; do not share code or the report across groups.

Use of AI coding assistants. Permitted; if used, briefly note in the report where and how.